

Study OS

A local-first, multimodal study platform — full project specification

Personal project (not affiliated with Starstreak), built together with a co-founder/friend. First target device: Redmi Note 13 Pro.

1. The Core Idea

Not a chatbot. A student gives the platform a question, a document, a photo, a recording, or a URL. The platform understands it, processes or researches whatever is necessary, reasons over it, and returns the most useful form of explanation it can: text, equations, graphs, diagrams, animations, interactive 2D/3D objects, PDFs, videos, or quizzes — whichever combination actually helps the student understand. Wherever possible, this happens entirely on the student's own device.

The distinction that matters: a chatbot answers questions. This platform is closer to a full study environment — it holds the student's material, remembers their progress, and chooses its own output format rather than always defaulting to a paragraph of text.

2. System Architecture — Five Layers

Flutter is the interface only — it contains no application logic. Python is the entire engine underneath it, and the LLM operates through that Python base rather than being a separate system bolted on top.

Layer	Responsibility
Flutter (interface)	Chat, notes, projects, camera, files, editor, graph/diagram/PDF/2D-3D viewers — rendering only.
Orchestrator (Python)	Understands the request, decides which engine(s) to call, combines results, constructs the structured response.
Local engines (Python)	OCR, PDF processing, mathematics, image processing, document generation, embeddings/RAG.
Local AI (LLM, via Python)	Runs through the Python base — routing, tutoring, summarization, explanation, quiz generation.
External world (optional)	Web search, web pages, YouTube, papers — reached only when device or local knowledge can't answer alone.

There is no Django requirement in this architecture — a centralized backend only becomes necessary later, if accounts, sync, or collaborative features are added.

3. How the Student Interacts

The student is never asked to pick a tool. The main screen is a single open prompt — “What are you studying?” — with attach, camera, and voice options alongside it, a list of recent projects (e.g. Thermodynamics, Fluid Mechanics, Engineering Mathematics), and a small set of quick actions (Scan, Research, Quiz, Notes).

From that one box, a student can type a direct question, attach a photo, upload a full textbook PDF, paste a YouTube URL, or say “I don't understand question 4.” The platform figures out what needs to happen next —

that decision is the orchestrator's job, not the student's.

4. Input Pipeline

Six kinds of input feed the platform, each handled differently before anything reaches the LLM:

- **Text** — a direct typed question.
- **Image** — camera capture, gallery pick, uploaded file, or screenshot.
- **PDF** — lecture notes, a textbook, a past paper, an assignment, a lab manual.
- **Web** — a URL, a search query, a website, or a YouTube video.
- **Audio** (later phase) — a spoken question or request.
- **Existing project material** — “my Fluid Mechanics project,” which lets the platform draw on everything already associated with that project.

5. Image Capture — Quality Analysis & Enhancement

A student's photo of a textbook page or whiteboard is routinely blurry, dark, tilted, noisy, low-contrast, or shot at an angle. The platform never sends a raw photo straight to OCR — it runs it through a quality-analysis and enhancement stage first, and always keeps the original image alongside the enhanced one so nothing is lost if enhancement guesses wrong.

Quality analysis checks for:

- Blur
- Noise
- Low resolution
- Poor lighting
- Perspective distortion
- Rotation
- Whether it's a document at all

Based on what it finds, adaptive enhancement applies whichever of these are needed:

- Deblur
- Denoise
- Sharpen
- Contrast correction
- Exposure correction
- Perspective correction
- Deskew
- Crop

Original image → Quality analysis → Adaptive enhancement → Enhanced image (original preserved)

6. Image Capture — Identifying What's Inside

Once an image is enhanced, a vision pass identifies what kind of content it actually contains, and routes it accordingly — this is what turns raw pixels into structured information the rest of the system can reason over.

Content type	Routed to
Ordinary text	OCR
Handwriting	Handwriting recognition
Mathematics	Math OCR → LaTeX
Graph	Graph / data extraction
Diagram	Diagram understanding
Photograph	Image understanding
Table	Structured table extraction

7. Worked Example — Photographing an Equation

EXAMPLE

Student photographs: $F = ma$, then asks “Explain how this applies to this problem.”

Camera → Image enhancement → Equation detection → Math OCR → LaTeX → Mathematical representation → + student's question → Reasoning over both

The system now has a structured equation and the student's request together, rather than a photo and a hope.

8. Worked Example — Photographing Lecture Notes

EXAMPLE

Student photographs a page beginning “Entropy is a thermodynamic property...”

Camera → Document detection → Perspective correction → Enhancement → OCR → Text structure extraction → AI

The photograph has effectively become a searchable study document. The student can now follow up with any of:

- “Summarize this.”
- “Turn this into revision notes.”

- “Generate 15 questions.”
- “Explain the second paragraph.”

9. PDFs as First-Class Citizens

A student uploads, say, **Thermodynamics.pdf**. The platform runs a full document engine over it — not just “extract the text” — pulling out metadata, text, page structure, images, tables, and equations, and running OCR on any scanned pages. The result becomes part of the student's local knowledge base, not just a file sitting in storage.

Once processed, the student can ask things like:

- “Summarize chapter 3.”
- “Find everything about entropy.”
- “Explain pages 42–47.”
- “Answer question 8.”
- “Generate flashcards from this chapter.”
- “Create a two-page revision sheet.”
- “Make me a practice exam.”
- “What concepts am I likely to struggle with?”

10. Local Document Intelligence (RAG)

Processed material is chunked, embedded, and stored in a local vector index. When the student asks a question, the system doesn't dump the entire PDF into the LLM's context — it embeds the question, searches the local index for relevant passages, and only feeds those to the model. This is what makes a small on-device model genuinely useful against long documents.

PDF → Text extraction → Chunking → Embeddings → Local vector index

Question → Embedding → Search local knowledge → Relevant passages → Local LLM → Answer

11. The Research Engine

For a question the local material can't answer — “Find recent research on hydrogen storage” — the platform activates a goal-directed research engine rather than blindly crawling the internet.

Question → Intent analysis → Needs external info? → Search → Discover sources → Fetch pages → Parse content → Clean content → Rank sources → Deduplicate → Extract useful info → Local AI → Answer + citations

12. External Sources Beyond Websites

The idea: don't make the student search for the explanation if the platform can find useful learning material for them. Sources aren't limited to text pages:

- **Websites** — research articles, documentation, educational sites, university resources.
- **YouTube** — relevant educational videos.
- **Papers** — academic literature.
- **Images** — useful diagrams or reference material.
- **Interactive resources** — simulations and educational tools.

13. YouTube as a Learning Modality

EXAMPLE

Student asks: “I don’t understand how a four-stroke engine works.” The platform can respond with an explanation, an interactive engine diagram, an animated four-stroke cycle, *and* a recommended video — with a one-line note on why that video specifically helps (e.g. “shows the four stages visually”), plus sources.

The video never replaces the explanation — it’s an additional modality alongside it.

14. Visual Explanation Engine

- **Static images** (PNG / JPEG / WebP).
- **Diagrams** — structured and editable, not decorative.
- **Interactive 2D** (SVG / canvas / native Flutter rendering).
- **Animations** — a process shown step-by-step.
- **Interactive 3D** (GLB / glTF) — a student can rotate and inspect an engine, a heart, a molecule, a machine, a building, an anatomical structure.

15. Animation

EXAMPLE

Student asks: “How does a transformer work?” The system builds a structured, animated sequence rather than a generic AI-generated clip:

AC input → Alternating magnetic field → Magnetic flux through core → Changing flux in secondary → Induced voltage

The distinction matters: this animation is structured and educational, built from the actual physical sequence — not a random generative-video output.

16. Interactive Simulations

The student experiments rather than just reads:

- **Physics** — change the projectile angle.
- **Engineering** — increase pressure and watch the piston.
- **Mathematics** — move a coefficient and watch the function change.
- **Chemistry** — change concentration and observe the effect.

Concept → Simulation model → Variables → Renderer → Interactive Flutter UI

17. Mathematics Engine

Mathematics gets its own deterministic engine rather than being left to the LLM: arithmetic, algebra, calculus, differentiation, integration, equations, matrices, statistics, probability, numerical methods, symbolic manipulation, unit conversion, and graphing.

EXAMPLE

Student asks: “Plot the function and explain its behavior.”

Equation → Symbolic representation → Graph engine → Interactive graph

The LLM explains the result; the deterministic engine performs the actual calculation — far safer than asking a small model to compute anything itself.

18. Engineering & Science Simulations

EXAMPLE

Student asks: “Show me Newton's second law.” Instead of just explaining it, the platform provides an interactive model where mass and force can be adjusted directly.

The same principle extends across domains:

- Mechanics
- Thermodynamics
- Fluid mechanics
- Circuits
- Electronics
- Structural mechanics
- Chemistry
- Statistics
- Economics

The simulation engine itself stays deterministic; the AI explains what's happening as the student changes variables.

19. Generating Documents (PDF Output)

The output engine also works in reverse — e.g. “Create revision notes from these 80 pages.”

Source material → Understand → Extract important concepts → Structure → Generate notes → Add equations → Add diagrams → Add tables → Generate PDF

Possible outputs:

- Revision notes
- Formula sheets
- Study guides
- Lab reports
- Summaries
- Practice exams
- Flashcard PDFs
- Printable worksheets

20. The Response Protocol

Instead of returning a single block of text, the orchestrator returns structured educational “blocks.” Flutter receives the block list and renders each one with the matching widget. This is the core communication contract between the Python engine and the Flutter interface.

Block type	Rendered as
text	Text widget
equation	Math renderer
interactive_graph	Graph widget
diagram	Diagram renderer
animation	Animation player
3d	3D viewer
video	Video player
pdf	PDF viewer
quiz	Quiz UI
table	Table UI
source	Source card

A single response can combine several blocks — e.g. a text explanation, an equation, an interactive graph, and two source cards, all in one reply.

21. Engineering Principle: the LLM Is Not the Brain of Everything

The single most important rule in this architecture: the LLM orchestrates and explains — it does not calculate, extract, search, or generate documents itself. Each of those has a deterministic engine underneath it, which is both more reliable and far cheaper to run on a small on-device model than asking the model to do everything.

Task	Handled by
Calculate	Math engine (sympy / numpy)
Extract PDF text	PDF parser (PyMuPDF)
Read a photo	OCR / vision pipeline
Search the internet	Search / crawler engine
Draw a graph	Graph engine
Explain the graph	LLM
Generate a quiz	LLM + deterministic validation

Generate a PDF	Document engine
----------------	-----------------

22. Local AI Strategy & Model Router

Inference runs via **llama.cpp** (not Ollama — a great desktop tool, but not the right mobile runtime), using quantized GGUF models. Rather than one model doing everything, the plan is a small router of models sized for what they're actually asked to do:

Tiny model (~0.3–0.6B): classification, routing, extraction → Main model (~1–2B): tutoring, Q&A, summaries → Specialized models: OCR, vision, speech, embeddings

Shortlist to benchmark against real tasks — not leaderboard scores:

Model	Size	Candidate role
Qwen3 0.6B	0.6B	Ultra-light routing / basic tutoring
Qwen3 1.7B	1.7B	Main local tutor candidate
Llama 3.2 1B	1B	General local assistant
Gemma 3 1B	1B	General tutoring / reasoning candidate
SmolLM2	135M–1.7B	Lightweight general assistant
LFM2 / LFM2.5	~1B	Routing / tool-calling experiments
Phi family	varies	Heavier reasoning / coding tasks

Benchmark criteria: explaining university-level concepts, extracting structured answers, summarizing PDFs, generating quizzes, following the block-response protocol reliably, tool selection accuracy, and real latency/RAM behavior on the actual target hardware — not a leaderboard score.

23. Local Knowledge Base

Every student's work becomes part of a personal, project-organized knowledge space:

- **Projects** (e.g. Thermodynamics, Fluid Mechanics) — each holding its own PDFs, notes, images, questions, and sources.
- **General knowledge** — material not tied to a specific project.

This structure is what makes contextual questions work — e.g. “Using the notes I've uploaded, explain entropy” — the platform searches the student's own material first, before ever reaching for the internet.

24. Personalization Over Time

Over time the platform can track subjects, current topics, previous questions, weak areas, notes, uploaded documents, quiz performance, and study history — which makes requests like these possible:

- “Quiz me on what I struggled with yesterday.”
- “Give me five questions on the concepts I haven't mastered.”

That's meaningfully more useful than a stateless, generic chatbot.

25. Study Modes

Different modes of the same engine — not separate apps, and not separate models:

Mode	What it does
Learn	Explain a concept
Solve	Work through a problem step-by-step
Research	Find and synthesize external sources
Practice	Generate exercises
Quiz	Test the student
Revise	Condense material
Create	Generate notes, PDFs, diagrams, graphs
Explore	Interactive simulations and 3D models

26. Voice (Later Phase)

Planned pipeline for spoken interaction:

Microphone → Speech recognition → Intent → Knowledge / AI → Response → Text + voice

Also supports requests like “Read this chapter to me,” with speech generated locally or through an optional service.

27. Offline-First Design

The internet is an extension, not the foundation.

Works fully offline	Requires internet
Notes, PDFs, OCR, image enhancement, mathematics, graphing, local AI, local LLM, quizzes, flashcards, existing sources, local projects	Web search, web LLM, quizzes, flashcards, existing sources, local projects

28. Cloud AI as an Optional Fallback

Local vs. cloud isn't a philosophical either/or — an AI router can decide per-request, based on whether the device is capable enough and whether the task is complex enough, always with the user's permission for anything leaving the device.

- “Summarize my three-page notes.” — Local.
- “Explain this simple equation.” — Local.
- “Analyze 400 pages of material with complicated reasoning.” — Potentially cloud, if the user permits it.

Cost model: bring-your-own-key (BYOK). Cloud LLM calls are not funded or proxied by the platform itself — a user who wants cloud fallback adds their own API key for whichever provider they choose (e.g. OpenAI, Anthropic, Google) in settings, and requests routed to the cloud are billed directly to that key. The platform incurs no per-request API cost of its own; it only sends requests using a key the user has supplied and can remove at any time. Local inference remains the default with no key required — BYOK only matters the moment a request is routed to the cloud tier.

This gives the platform graceful degradation instead of a hard capability ceiling.

29. Privacy & Security

Students upload assignments, personal notes, and sometimes unpublished work — so local processing is a real privacy advantage, not just a performance choice. The interface should make it clear, per action, whether something stays on-device (■■ On device — your document stays on your phone) or requires external access (■■ Online — this requires external access), so the student always knows when data leaves the device.

30. Local Storage Design

SQLite holds structured data: users, projects, conversations, notes, documents, sources, questions, quiz results, study history. The filesystem holds large objects — PDFs, images, audio, generated files, model

weights — under a per-project folder structure (e.g. /storage/projects/thermodynamics/{pdfs, images, audio, generated, models}). A local vector index sits alongside both for semantic search over the student's own material.

31. Technical Architecture Mapping

Layer	Responsible for
Flutter	UI, navigation, state, animations, editor, camera UI, file UI, PDF viewer, graph viewer, 2D rendering, 3D viewer, i
Python	Web scraping, crawling, HTML parsing, PDF processing, document processing, mathematics, data processing, i
Native Android/iOS	Camera, file system, GPU acceleration, native ML inference, LLM runtime, background execution, platform APIs
Local AI	llama.cpp / MLC / ONNX Runtime → quantized SLM → on-device inference.
Storage	SQLite + local filesystem + vector index.

32. Full Request Lifecycle — Worked Example

EXAMPLE

Student takes a blurry photograph of a question and asks: “Solve this and explain it to me.”

- 1 Camera captures the image
- 2 Quality analysis runs
- 3 Deblur / denoise / sharpen applied
- 4 Perspective correction applied
- 5 Document/question detected
- 6 OCR runs
- 7 Mathematical recognition runs
- 8 A structured problem is produced
- 9 Intent is determined
- 10 The mathematical engine solves it
- 11 The local LLM explains the solution
- 12 The response planner decides which blocks to include (explanation, equations, graph, worked steps)
- 13 Flutter renders everything
- 14 The student asks a follow-up, context is retained
- 15 The student saves it to a project
- 16 A PDF is generated if requested

If the question needs current information instead, the path branches into the research pipeline:

Question → Intent analysis → Needs web data? → Search engine → Crawler → Source extraction → Source ranking → Local knowledge → LLM → Text + sources + video + visuals

33. The Core Design Decision

This is deliberately **not** “Flutter frontend + Python backend” — that framing is too simplistic for what this needs to do. It’s better described as: **Flutter experience layer + local computational engines + local AI + optional internet intelligence**. Python is one of the major engines, not necessarily the entire engine. And the LLM is not the entire brain either.

The real brain is the orchestrator, which works through this sequence for every request:

```
What did the student give me? → What are they asking? → What information do I need? → Which  
engine should handle it? → Do I need the internet? → Do I need AI? → What should the answer look  
like? → How should Flutter render it?
```

34. Target Hardware

First target device: **Redmi Note 13 Pro** (4G / Helio G99 Ultra variant assumed — 2× Cortex-A76 @2.2GHz + 6× Cortex-A55 @2.0GHz, Mali-G57 MC2, 8/12GB RAM, MIUI/HyperOS).

- No dedicated NPU worth targeting on this chipset — inference runs on the CPU via llama.cpp's ARM NEON path, mainly the two big A76 cores.
- 8–12GB RAM gives real headroom: a Q4_K_M quantized 1.7B model (~1.1–1.3GB on disk) should run comfortably; 3–4B is plausible for latency-tolerant tasks like note generation.
- MIUI/HyperOS aggressively kills background processes — this affects packaging decisions, not just battery life.

35. Bundling & Distribution

Because this will be shared with a friend rather than run only on one known device, the platform needs to ship as a single installable app — not a manually-configured setup (an earlier plan using Termux as an external Python host was rejected for exactly this reason: too fragile to hand to someone else).

- **Chaquopy** embeds a real CPython interpreter inside the Android app process itself, so the whole thing ships as one APK: Flutter → platform channel → Android host (Kotlin) → Chaquopy → Python engine.
- **LLM inference specifically** runs through a native llama.cpp build (compiled via the NDK) rather than through *llama-cpp-python*, since that package isn't a clean cross-compile under Chaquopy. Python calls into it via Chaquopy's Java interop → Kotlin → JNI → native llama.cpp. Python still decides when to generate and what to feed the model, and still receives and routes the output — the architectural intent holds even though the raw matrix math happens in native code (exactly what *llama-cpp-python* does on desktop anyway).
- **Model weights are not bundled in the APK.** They're downloaded on first launch and stored in app-private storage, keeping the installer small and easy to share while the app stays fully offline-capable afterward.
- **A first-run capability check** (available RAM, quick CPU benchmark) picks the model size, since friends' devices won't all match the Redmi Note 13 Pro's specs.
- **MIUI/HyperOS specifically** requires disabling battery optimization and enabling autostart for the app once background inference or persistent state is involved.

36. Recommended Build Sequence

- 1 **Prototype the Python engine on Termux first.** FastAPI + llama-cpp-python + a quantized GGUF model, one /chat endpoint, benchmark real tokens/sec on-device before building anything else. Far faster to iterate on than debugging inside a Chaquopy-embedded build.
- 2 **Get native llama.cpp inference working standalone** (NDK build, JNI wrapper) — the highest-risk piece of the bundled version, worth de-risking early.

- 3 **Scaffold the Chaquopy + Flutter project structure** and port the proven orchestrator logic over, swapping the inference call from *llama-cpp-python* to the native-lib path.
- 4 **Build outward from the request-lifecycle MVP loop** — camera → enhancement → OCR → math engine → local LLM explanation → rendered response — before touching animation, interactive 3D, or simulations, which are a large amount of engineering for comparatively little differentiation at this stage.

37. Open Questions

- Confirm the exact Redmi Note 13 Pro chipset variant (Helio G99 Ultra vs. the Snapdragon 7s Gen 2 5G variant) — changes inference speed expectations.
- Decide the structured-output validation/repair strategy for small-model JSON reliability (schema validation + retry, or grammar-constrained decoding via GBNF).
- Decide how much of animation / 3D / simulations is in scope for a first shared version vs. a later phase.

Prepared as a working document for project discussion between co-founders/collaborators.